

Indexing (Examples)

[For details and different example, you must read the previous note first]

Indexing in DBMS is a technique that improves the speed of data retrieval operations by using an auxiliary data structure that allows quick access to records without scanning the entire table.

Why indexing is needed?

Without an index:

- The database performs a **full table scan**
- Time complexity $\approx O(n)$

With an index:

- Database directly locates data
- Time complexity $\approx O(\log n)$ (usually)

Disadvantages

- Extra **storage space**
- Slower **INSERT, UPDATE, DELETE** operations (index must be updated)

How indexing works

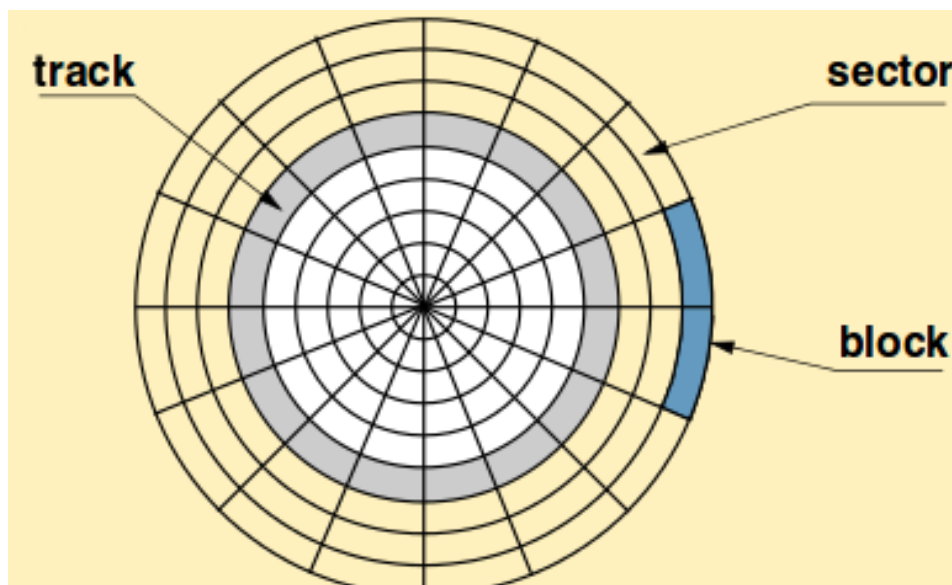
- An index stores **key values** (from one or more columns)
- Each key points to the **location (address)** of the actual row in the table
- Indexes are commonly implemented using:
 - **B-tree / B+ tree**
 - **Hash tables**

Types of indices:

1. **Ordered indices:** Based on a sorted ordering of the indexed key values.
2. **Hash indices:** Based on a uniform distribution of indexed key values (determined by hash function) across arrange of buckets.

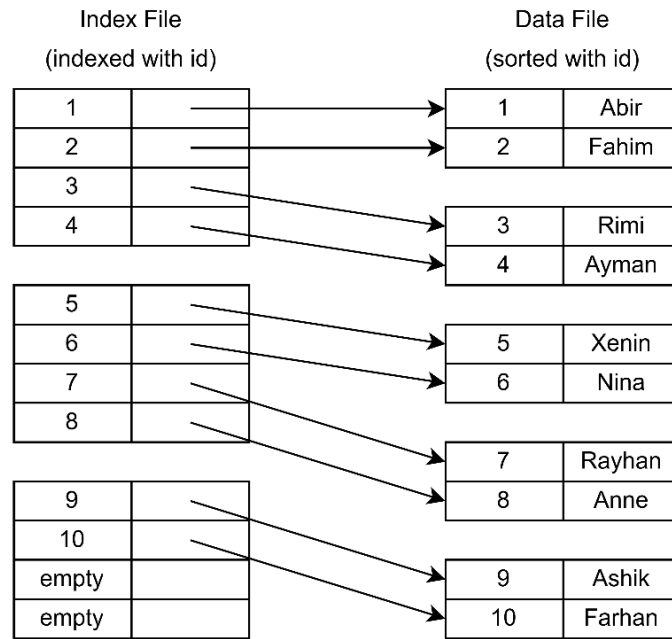
You must read the previous hand-note to understand these:

- All the example here Assumed –
 - Index record per block: 4
 - Data Record per block: 2



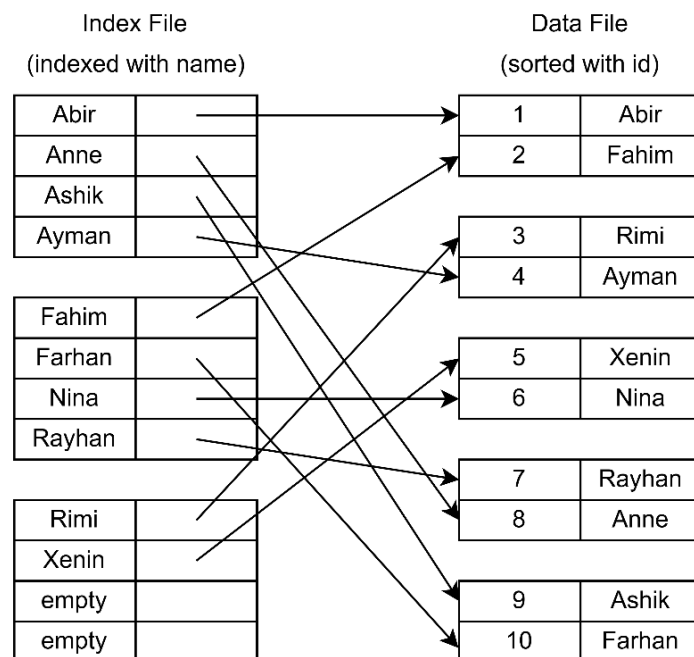
Disk Block: Could contain 2 or more block together.

- 1) **Primary Index/ Clustering index:** Primary indexing is an indexing technique where an index is created on the primary key of a sorted data file to allow fast record retrieval.



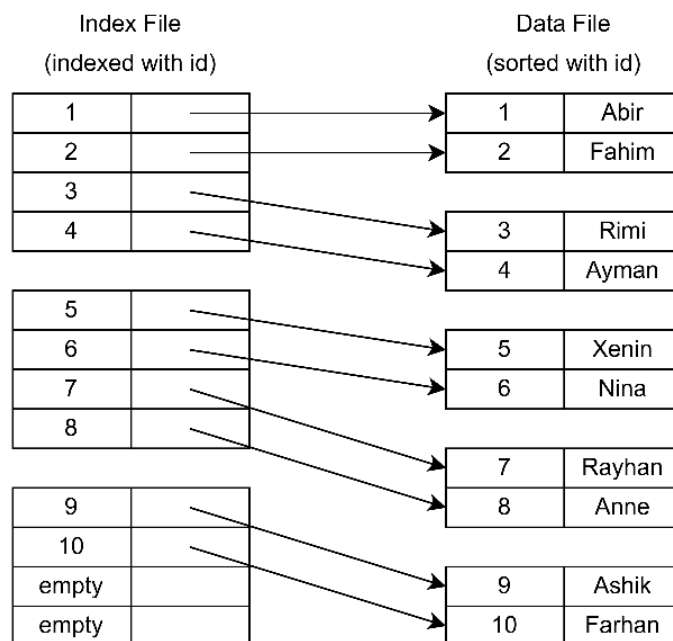
Primary Index

- 2) **Secondary Index / Non-clustering Index:** A secondary (non-clustered) index is an index created on a non-primary attribute to speed up data retrieval when the data file is not physically ordered on that attribute.



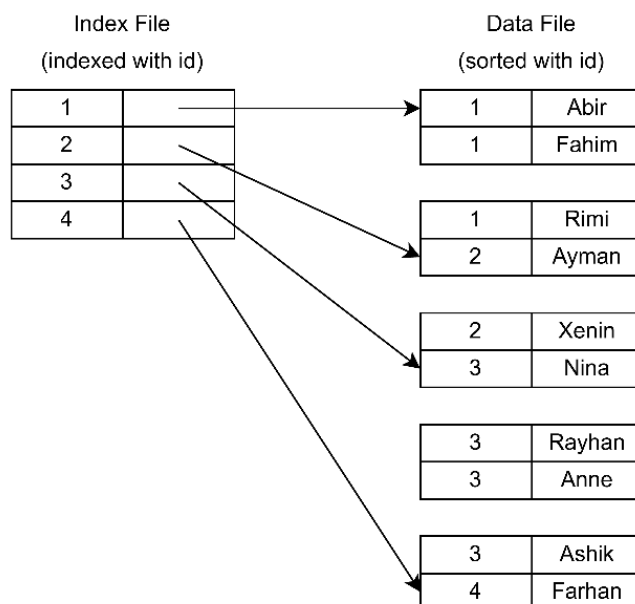
Secondary Index

- 3) **Dense + Primary Index + No duplicates:** A dense primary index is an index where every unique primary key value has a corresponding index entry pointing directly to its record.



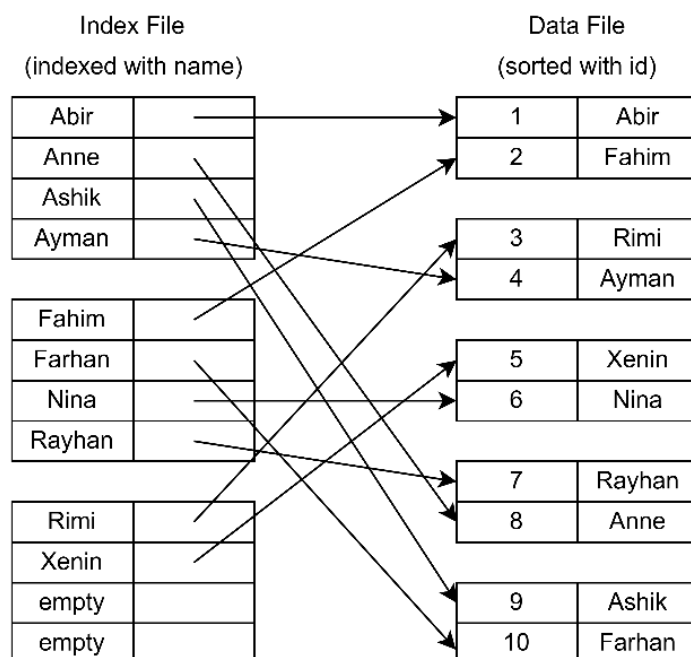
Dense + Primary Index
(No Duplicates)

- 4) **Dense + Primary index + with duplicates:** A dense primary index with duplicates is an index where every search key value has an index entry, and duplicate key values point to multiple records through a list of pointers. Even though it is called a *primary index* in some textbooks, duplicates are allowed because the index is built on the ordering field, not a strict primary key.



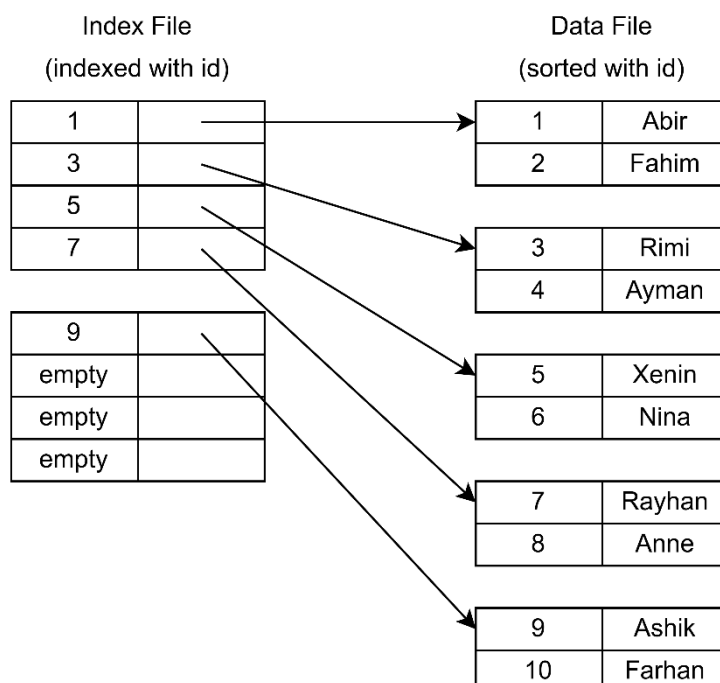
Dense + Primary Index
(With Duplicates)

- 5) **Dense + Secondary Index (with/without Duplicate Search Keys):** A dense secondary index is an index created on a non-primary attribute where every search key value has an index entry, pointing to one or more records depending on whether duplicates exist.



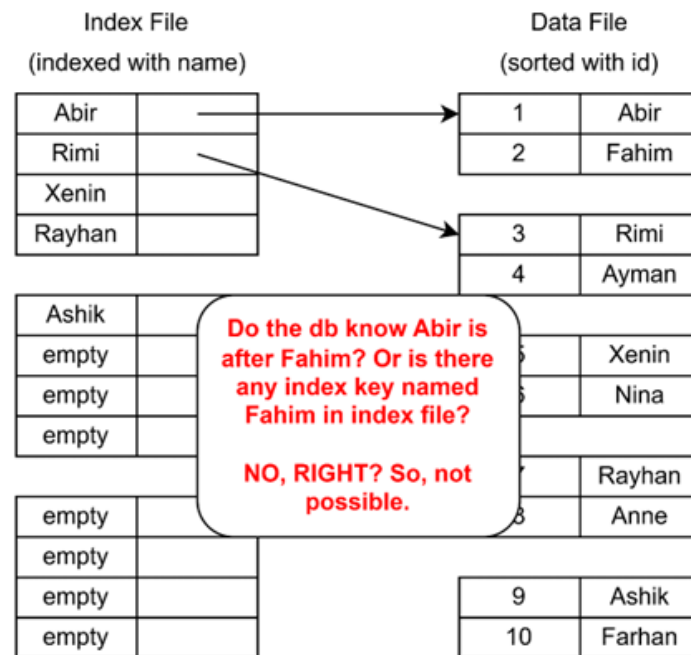
Dense + Secondary Index
(with/without Duplicate
Search Keys)

- 6) **Sparse + Primary Index:** A sparse primary index is an index on a sorted primary key where index entries point to data blocks instead of individual records, reducing storage overhead.



Primary + Sparse Index

- 7) **Sparse + Secondary Index (Not Possible):** Sparse secondary index is not possible because secondary indexes are built on unsorted data, whereas sparse indexing requires sorted data.



Secondary + Sparse Index
(Not Possible) - As name aren't
sorted in HDD File.

- 8) **Multilevel Index:** A multilevel index is an index structure where higher-level indexes point to lower-level indexes, ultimately leading to the data, used to efficiently handle large index files.

